



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Outline

Part 0. Overview

Part 1. Software Processes

Part 2. Modeling

Part 3. Quality Assurance

Part 4. Software Project Management

- ❖ **Chap 22. Project Management Concepts**
- ❖ **Chap 23. Process and Project Metrics**
- ❖ **Chap 24-25. Project Estimation & Scheduling**
- ❖ **Chap 26. Risk Analysis**

Chap 24-25. Project Estimation & Scheduling

- ❖ **Project Planning Tasks**
- ❖ **Establish project scope**
- ❖ **Estimate cost and effort**
 - **Line Of Code**
 - **Function Point**
 - **COCOMO I**
 - **COCOMO II**
- ❖ **Scheduling principle**

Software Project Planning

The overall goal of project planning is to establish a pragmatic strategy for controlling, tracking, and monitoring a complex technical project.

Why?

*So the end result gets done on time,
with quality!*

软件项目规划

项目规划的总体目标是建立一种务实的策略，用于控制、跟踪和监控复杂的技术项目。

为什么？

为了确保最终结果按时完成，且具有高质量！

Project Planning Task Set-I

❖ **Establish project scope**

❖ **Determine feasibility**

❖ **Analyze risks**

❖ **Define required resources**

- **Determine required human resources**

- **Define reusable software resources**

- **Identify environmental resources**

- 确定项目范围

- 定义所需资源

- 确定可行性

- 确定所需的人力资源

- 定义可重用的软件资源

- 分析风险

- 确定环境资源

Project Planning Task Set-II

❖ Estimate cost and effort

- Decompose the problem
- Develop two or more estimates using size, function points, process tasks or use-cases
- Reconcile (核对) the estimates

❖ Develop a project schedule

- Establish a meaningful task set
- Define a task network
- Use scheduling tools to develop a timeline chart
- Define schedule tracking mechanisms

• 估算成本和工作量

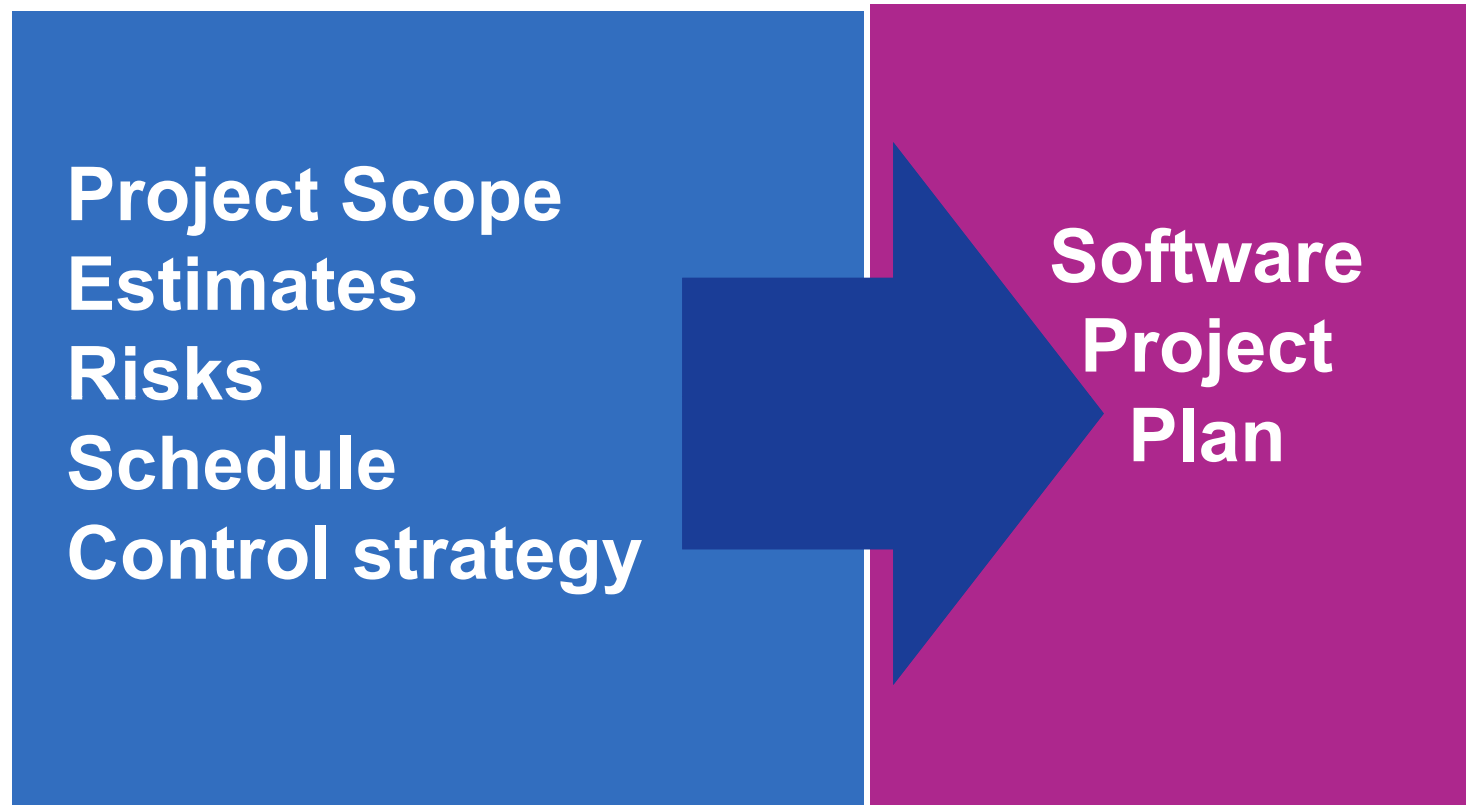
- 分解问题
- 使用规模、功能点、过程任务或用例进行两个或更多估算
- 核对估算

• 制定项目进度表

- 建立有意义的任务集
- 定义任务网络

- 使用调度工具开发时间线图
- 定义进度跟踪机制

Write it Down!



To Understand Scope ...

- ❖ Understand the customers needs
- ❖ understand the business context
- ❖ understand the project boundaries
- ❖ understand the customer's motivation
- ❖ understand the likely paths for change
- ❖ understand that ...

*Even when you understand,
nothing is guaranteed!*

- 了解客户需求
- 了解客户的动机
- 了解业务背景
- 了解可能的变化路径
- 了解项目边界
- 了解...

即使你理解了，什么也无法保证！

What is Scope?

- ❖ **Scope** refers to all the work involved in creating the products of the project and the processes used to create them. **It defines what is or is not to be done**

范围是指创建项目产品和用于创建它们的过程所涉及的所有工作。它定义了哪些是或不是要完成的工作。

- ❖ *Software scope* describes

- the **functions and features** that are to be delivered to end-users
- the **data** that are input and output
- the **“content”** that is presented to users as a consequence of using the software
- the **performance, constraints, interfaces, and reliability** that *bound* the system.

软件范围 描述:

- 交付给最终用户的功能和特性
- 作为使用软件的结果呈现给用户的“内容”
- 输入和输出的数据
- 约束系统的性能、限制、接口和可靠性

What is Scope?

❖ Scope is defined using one of two techniques:

- A narrative description of software scope is developed after communication with all stakeholders.
- A set of use-cases is developed by end-users.

❖ The project team and stakeholders must have the same understanding of

- What products will be produced as a result of a project?
- How they'll be produced?

This needs good communication!

范围使用两种技术之一定义：

项目团队和利益相关者必须对以下内容有相同的理解：

- 在与所有利益相关者沟通后开发的软件范围的叙述描述。
- 项目将产生哪些产品？
- 由最终用户开发的一组用例。
- 这些产品将如何生产？

这需要良好的沟通！

工作分解结构 (WBS)

Work Breakdown Structure (WBS)

❖ A work breakdown structure is a deliverable-oriented grouping of the work involved in a project that **defines the total scope of the project.**

- 工作分解结构是一个以可交付成果为导向的分组，涵盖了项目中的所有工作，定义了项目的总体范围。

❖ It provides the basis for

- Planning and managing project schedules, costs, and changes
- Risk analysis
- Organizational structure
- Measurement

- 它提供了以下基础：

- 规划和管理项目时间表、成本和变更
- 风险分析
- 组织结构
- 衡量

WBS

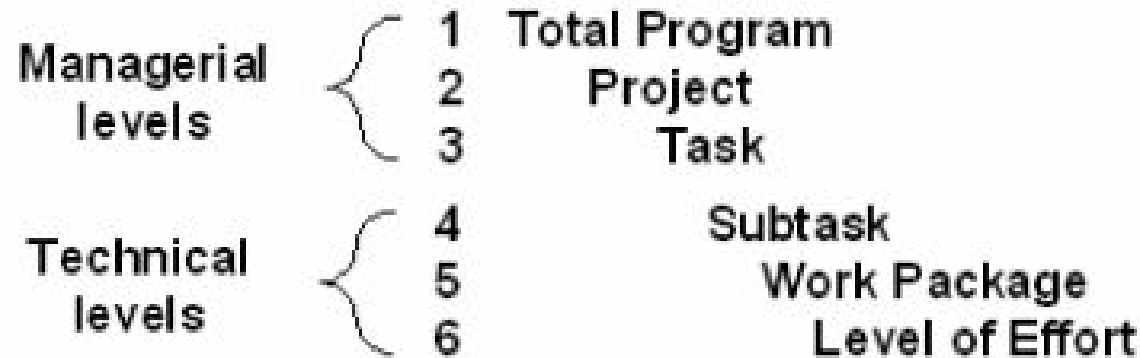
❖ A WBS may be a diagram (graphical tree) or a text list (outline format).

- Graphical Tree format: the project as the root node, major components of the project as the main branches, and tasks/subtasks branch off from these.
- Outline format: uses a decimal numbering system, E.g., 3.1.5

- WBS可以是图表（图形树）或文本列表（大纲格式）。
 - 图形树格式：项目为根节点，项目的主要组成部分为主分支，任务/子任务从这些分支扩展。
 - 大纲格式：使用十进制编号系统，例如3.1.5

A Full WBS Structure

❖ Up to six levels (3-6 usually) such as



❖ Upper 3 can be used by customer for reporting

❖ Different level can be applied to different uses

■ E.g., Level 1: authorizations;

■ 2: budgets;

■ 3: schedules

- 上三层可以用于客户报告

- 不同层次可以应用于不同用途:

- 例如, 第一层: 授权; 第二层: 预算; 第三层: 时间表

- 多达六个层次 (通常为3-6层), 例如:

- 管理层级: 1. 总项目, 2. 项目, 3. 任务

- 技术层级: 4. 子任务, 5. 工作包, 6. 努力程度

WBS Types

❖ WBS is usually organized around

- project products (tools, tangible outputs, .)
- project phases (concepts, steps,...)
- project management process groups (initializing, developing, executing, closing)

- WBS通常围绕以下组织：
 - 项目产品（工具、可交付成果等）
 - 项目阶段（概念、步骤等）
 - 项目管理过程组（初始化、开发、执行、关闭）

❖ Product WBS

- Entity-oriented
 - E.g., Financial engine, Interface system, DB
 - Typically used by engineering manager

• 产品WBS

- 实体导向，例如，金融引擎、接口系统、数据库
- 通常由工程经理使用

❖ Process WBS

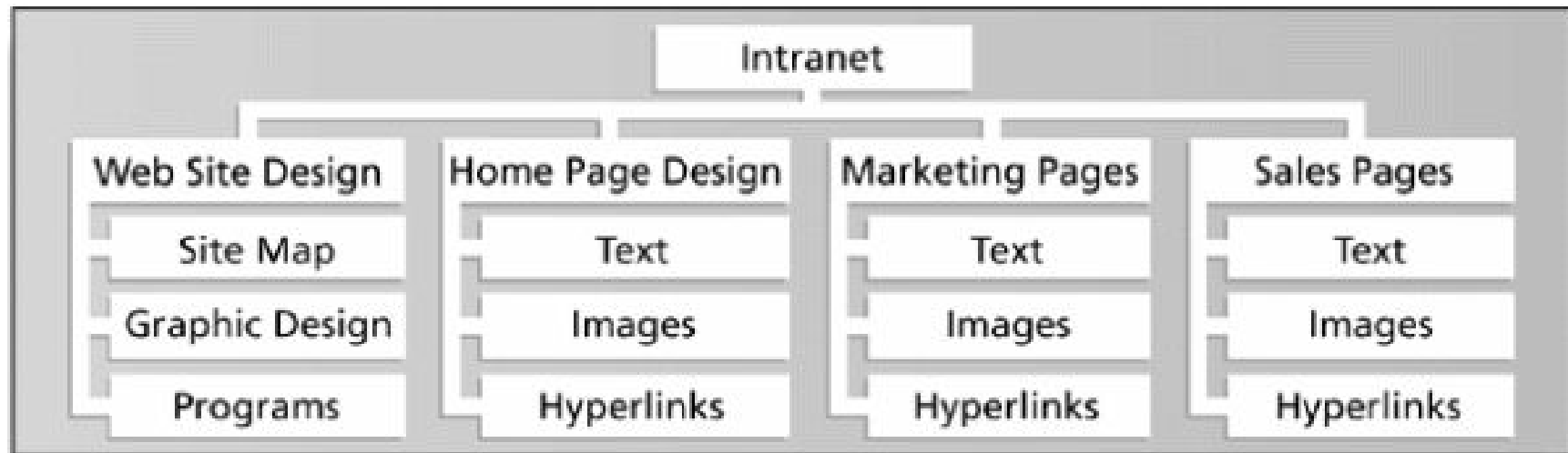
- Activity-oriented
 - E.g., Requirements, Analysis, Design, Testing
 - Typically used by project managers

- 活动导向，例如，需求、分析、设计、测试
- 通常由项目经理使用

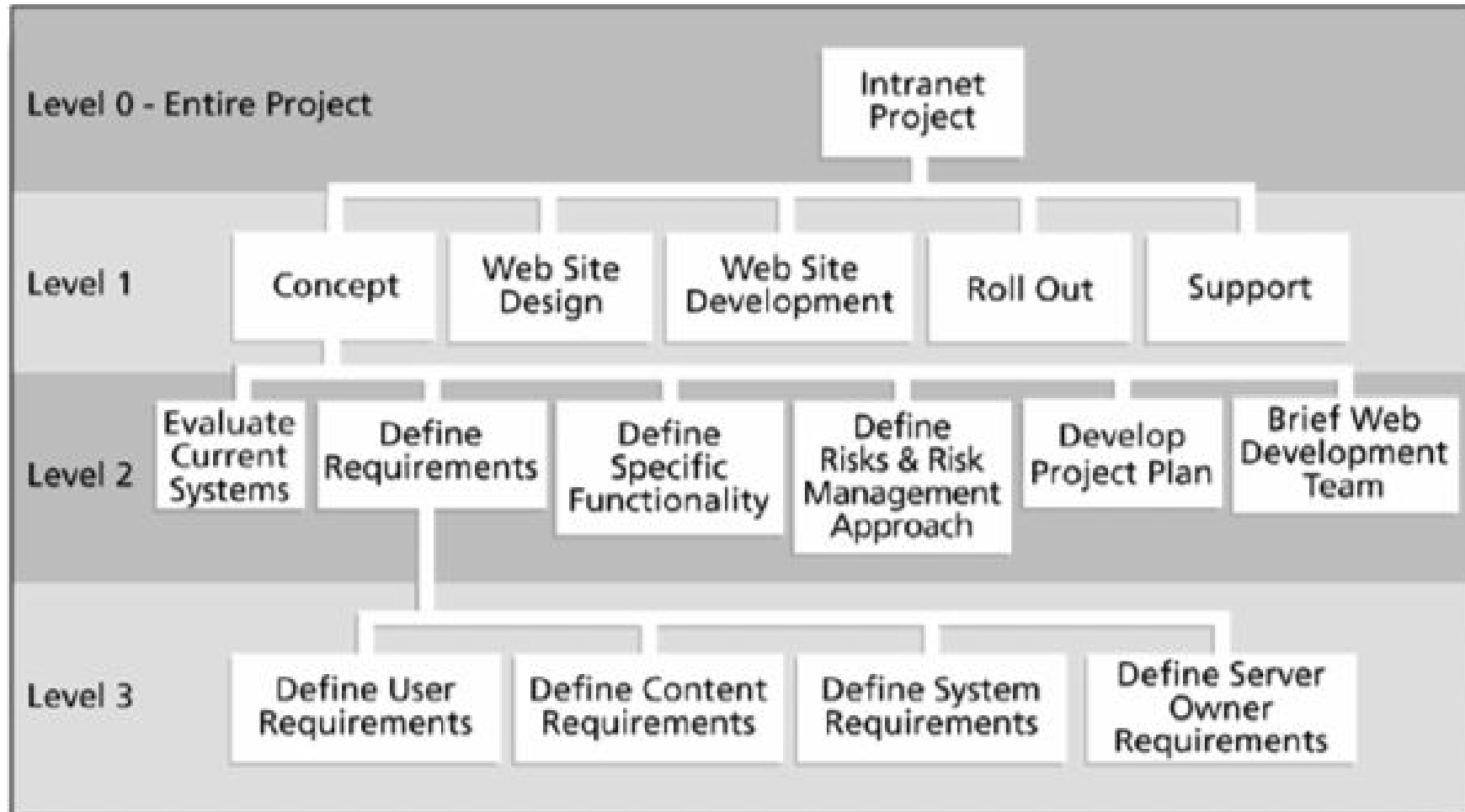
1. WBS的种类

- 产品WBS——面向实体
- 过程WBS——面向活动

Example: Product WBS



Example: Process WBS



之前的大纲格式WBS

Previous WBS in Outline Format

1.0 Concept

1.1 Evaluate current systems

1.2 Define Requirements

1.2.1 Define user requirements

1.2.2 Define content requirements

1.2.3 Define system requirements

1.2.4 Define server owner requirements

1.3 Define specific functionality

1.4 Define risks and risk management approach

1.5 Develop project plan

1.6 Brief Web development team

2.0 Web Site Design

3.0 Web Site Development

4.0 Roll Out

5.0 Support



Work Packages

- ❖ **Generic term for discrete tasks with definable end results**
 - ❖ **Typically the “leaves” on the WBS tree**
 - ❖ **Use the “one-to-two” rule**
 - 1 or 2 persons for 1 or 2 weeks
 - ❖ **Ideally shorter rather than longer**
 - 2-3 weeks maximum for software projects
 - 1 day minimum
 - Not so small as to micro-manage
 - ❖ **Basis for monitoring and reporting progress**
 - Can be tied to budget items (with charge numbers)
 - Resources (personnel) assigned
 - 使用“一到两”规则
 - 1或2人, 1或2周
 - 理想情况下较短, 而不是更长
 - 软件项目的最大时间为2-3周, 最短时间为1天
 - 不要太小以至于微管理
- 通用术语, 用于具有可定义最终结果的离散任务
 - 通常是WBS树上的“叶子”
 - 监控和报告进度的基础
 - 可以与预算项目 (带有费用编号) 相关联
 - 分配资源 (人员)

Approaches to Developing WBS

- ❖ **1. Using guidelines:** Some organizations provide guidelines for preparing WBSs
- ❖ **2. Analogy approach:** Review WBSs of similar projects and tailor to our project
- ❖ **3. Top-down approach:** Start with the largest items of the project and break them down
- ❖ **4. Bottom-up approach:** Start with the detailed tasks and roll them up
- ❖ **5. Mind-mapping approach:** Write down tasks in a non-linear format and then create the WBS structure

Approaches to Developing WBS

2. 进行WBS的五种方法

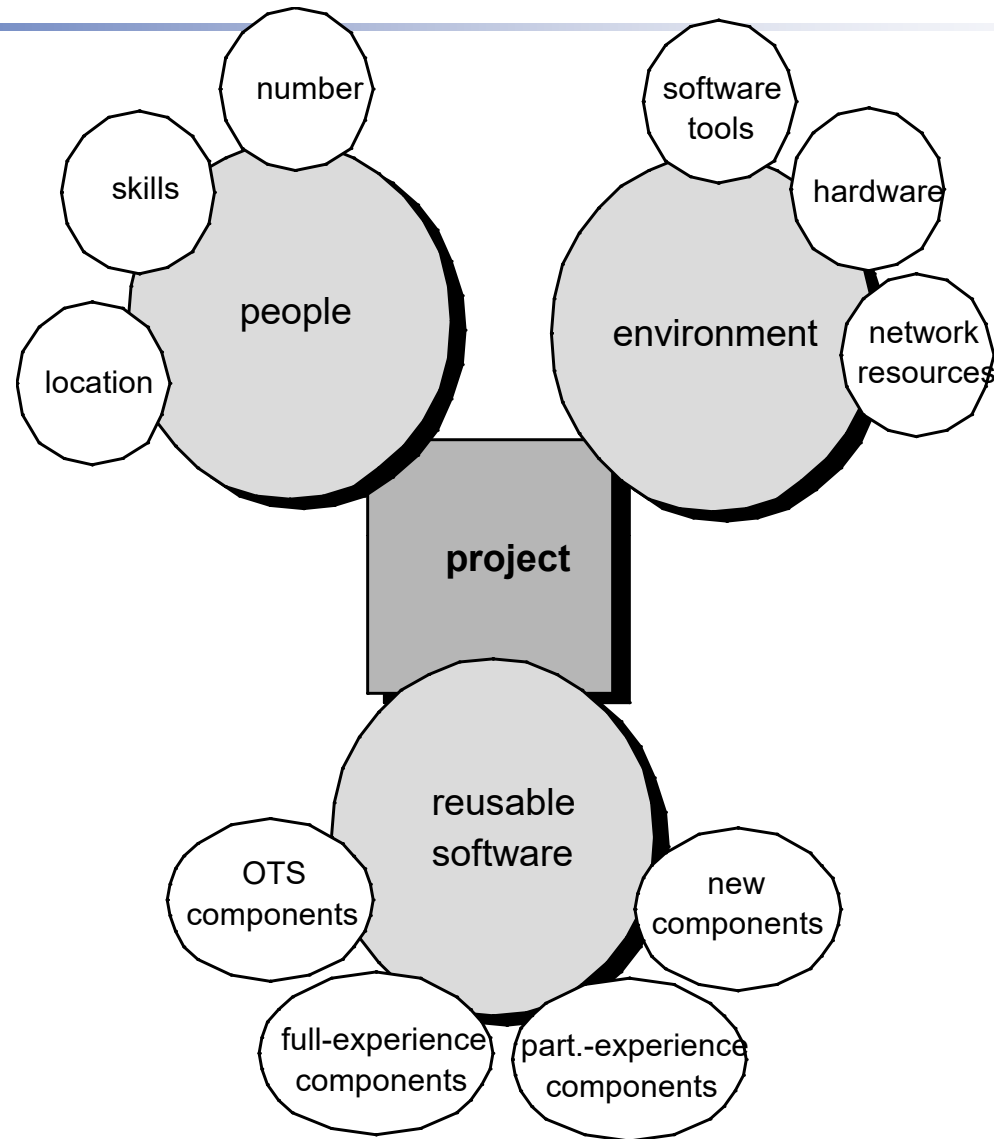
1. 使用指南：一些组织提供的指南
2. 类比方法：审查类似项目的 WBS，并根据我们的项目 进行定制。
3. 自上而下的方法：从项目中最大的项目开始，并将它们分解
4. 自下而上的方法：从详细的任务开始，并将它们汇总
5. 思维导图方法：以非线性格式写下任务，然后创建 WBS 结构

WBS

**If it's not in the WBS,
It's not in the project !**

如果它不在WBS中，它就不在项目中！

Resources



Project Estimation

项目估算



- ❖ **Project scope must be understood**
- ❖ **Elaboration (decomposition) is necessary**
- ❖ **Historical metrics are very helpful**
- ❖ **At least two different techniques should be used**
- ❖ **Uncertainty is inherent in the process**
 - 必须理解项目范围
 - 需要展开（分解）
 - 历史指标非常有帮助
 - 应该使用至少两种不同的技术
 - 不确定性是过程中固有的

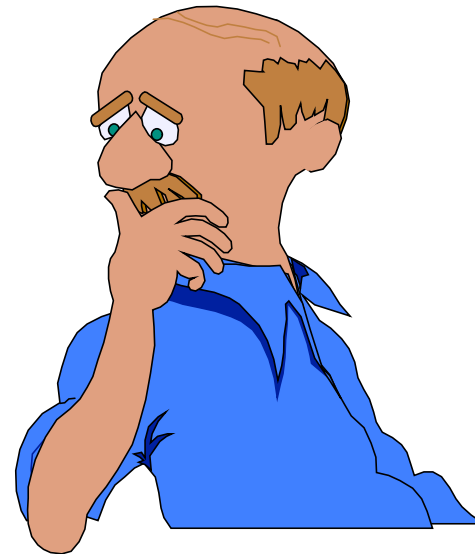
Estimation Techniques 估算技术

- ❖ **Past (similar) project experience**
- ❖ **Conventional estimation techniques**
 - Task breakdown and effort estimates
 - Size (e.g., FP) estimates

- ❖ **Empirical models**

- ❖ **Automated tools**

- 过去（类似）项目经验
- 传统估算技术
 - 任务分解和努力估算
 - 大小（例如，功能点）估算
- 经验模型
- 自动化工具



Estimation Accuracy

❖ Predicated on ...

- **the degree to which the planner has properly estimated the size of the product to be built**
- **the ability to translate** the size estimate into human effort, calendar time, and dollars (a function of the availability of reliable software metrics from past projects)
- the degree to which the project plan reflects the **abilities of the software team**
- the **stability of product requirements** and the environment that supports the software engineering effort.

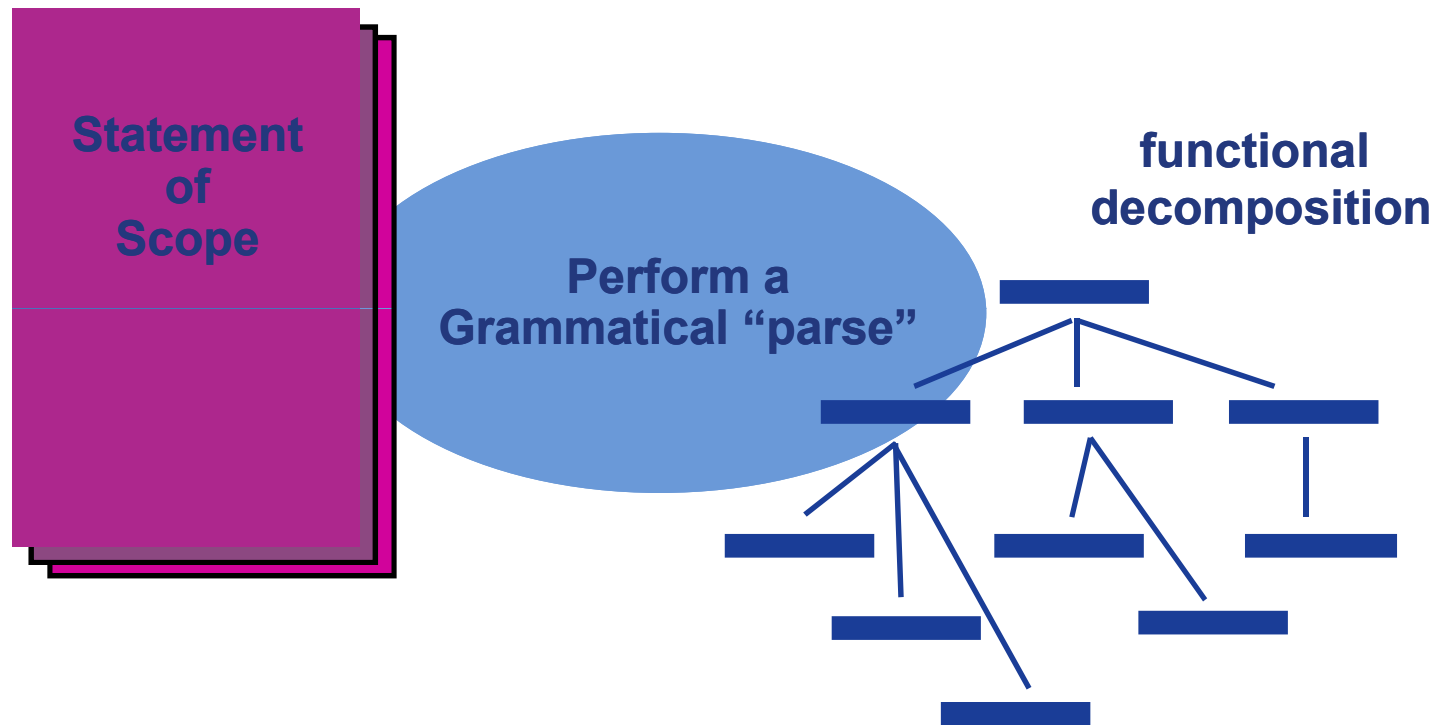
Estimation Accuracy

估算准确性

取决于.....

- 计划者是否正确估算了要构建产品的规模
- 将规模估算转换成人力、日历时间和成本的能力（取决于过去项目中可靠软件指标的可用性）
- 项目计划反映软件团队能力的程度
- 产品需求的稳定性和支持软件工程工作的环境

Functional Decomposition 功能分解



Conventional Methods: LOC/FP Approach

- ❖ **Compute LOC/FP using estimates of information domain values**
- ❖ **Use historical data to build estimates for the project**

常规方法：代码行/功能点方法

- 使用信息域值估算计算代码行/功能点
- 使用历史数据构建项目估算

Example: LOC Approach

Function	Estimated LOC
user interface and control facilities (UICF)	2,300
two-dimensional geometric analysis (2D GA)	8,300
three-dimensional geometric analysis (3D GA)	6,800
database management (DBM)	3,300
computer graphics display facilities (CGDF)	4,900
peripheral control (PC)	2,100
design analysis modules (DAM)	8,400
<i>estimated lines of code</i>	33,200

Example: LOC Approach

Function	S_{opt}	S_m	S_{pess}	$S = (S_{opt} + 4S_m + S_{pess})/6$
UICF	...	...	...	2300
2DGA	...	...	...	5300
3DGA	4600	6900	8600	6800
DBM	...	...	...	3350
...				
Estimated LOC	...	...	...	33200

Example: LOC Approach

- ❖ **Average productivity** for systems of this type = **620 LOC/pm.**
- ❖ **Burdened labor rate = \$8000 per month**, the cost per line of code is approximately \$13.
- ❖ Based on the LOC estimate and the historical productivity data, the total estimated project cost is **\$432,000** and the estimated effort is **54 person-months.**

LOC Approach

❖ Advantages

- Simple to measure
- Easy to automate

❖ Disadvantages

- Only defined for code, not the design
- Bad design may cause excessive LOC
- Language dependent
- Does not account for functionality or complexity

• 优点

- 易于衡量
- 易于自动化

• 缺点

- 仅针对代码定义，而非设计
- 糟糕的设计可能导致过多的代码行
- 语言依赖性
- 不考虑功能性或复杂性

Example: FP Approach

Information Domain Value	opt.	lkely	pass.	est. count	weight	FP-count
number of inputs	20	24	30	24	4	97
number of outputs	12	18	22	16	8	78
number of inquiries	16	22	28	22	8	88
number of files	4	4	8	4	10	42
number of external interfaces	2	2	3	2	7	18
count-total						321

Example: FP Approach

	Adjusting Factors	Fi (0-5)
1	data communication	2
2	on-line data entry	4
3	complex processing	5
4	operational ease	5
5	facilitate change	5
...	...	
14	...	
	Total	52

Example: FP Approach

- ❖ The estimated number of FP is derived:
- ❖ $\text{FP}_{\text{estimated}} = \text{count-total} * [0.65 + 0.01 * S (Fi)]$
- ❖
- ❖ $\text{FP}_{\text{estimated}} = 375$
- ❖ organizational average productivity = **6.5 FP/pm.**
- ❖ burdened labor rate = \$8000 per month, approximately \$1230/FP.
- ❖ Based on the FP estimate and the historical productivity data, total estimated project cost is **\$461,000** and estimated effort is **58 person-months.**

FP Approach

❖ Advantages

- Usable in the **earliest** requirements phases
- **Independent** of programming language, product design, or development style
- **User's view**, rather than implementation view
- Can be used to measure the **non-coding activities**
- There exists a large body of **historical data**
- A well **documented** method

- 优点
 - 可用于最早的需求阶段
 - 独立于编程语言、产品设计或开发风格
 - 用户视角，而非实现视角
 - 可用于衡量非编码活动
 - 存在大量历史数据
 - 方法文档详尽

FP Approach

❖ Disadvantages

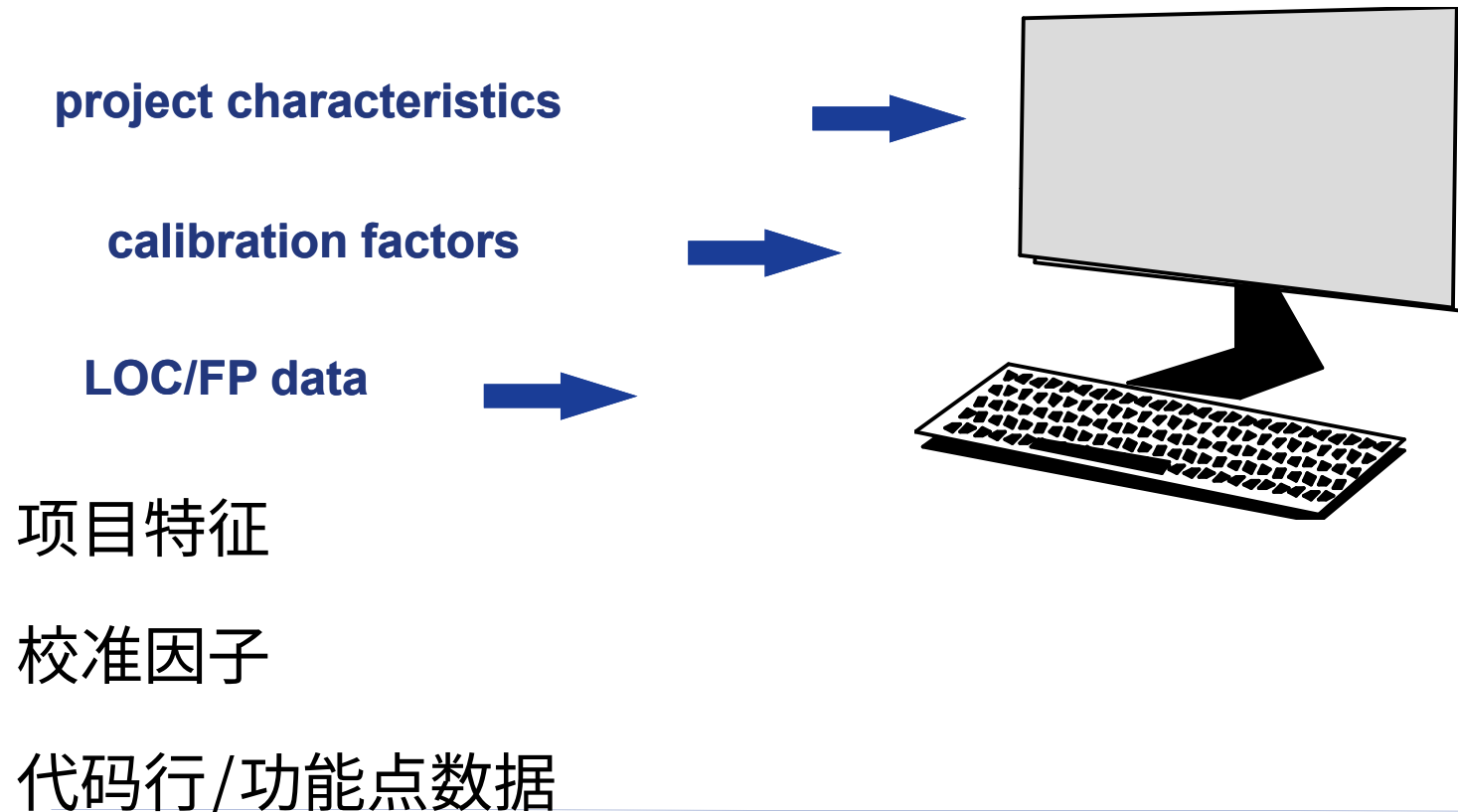
- Cannot directly count an existing product's (source code) FP content
- Hard to automate
- FP do not reflect language, design, or style differences
- FP are designed for estimating commercial data processing applications
- Subjective counting

• 缺点

- 不能直接计算现有产品的功能点内容
- 难以自动化
- 功能点不反映语言、设计或风格差异
- 功能点设计用于估算商业数据处理应用
- 主观计数

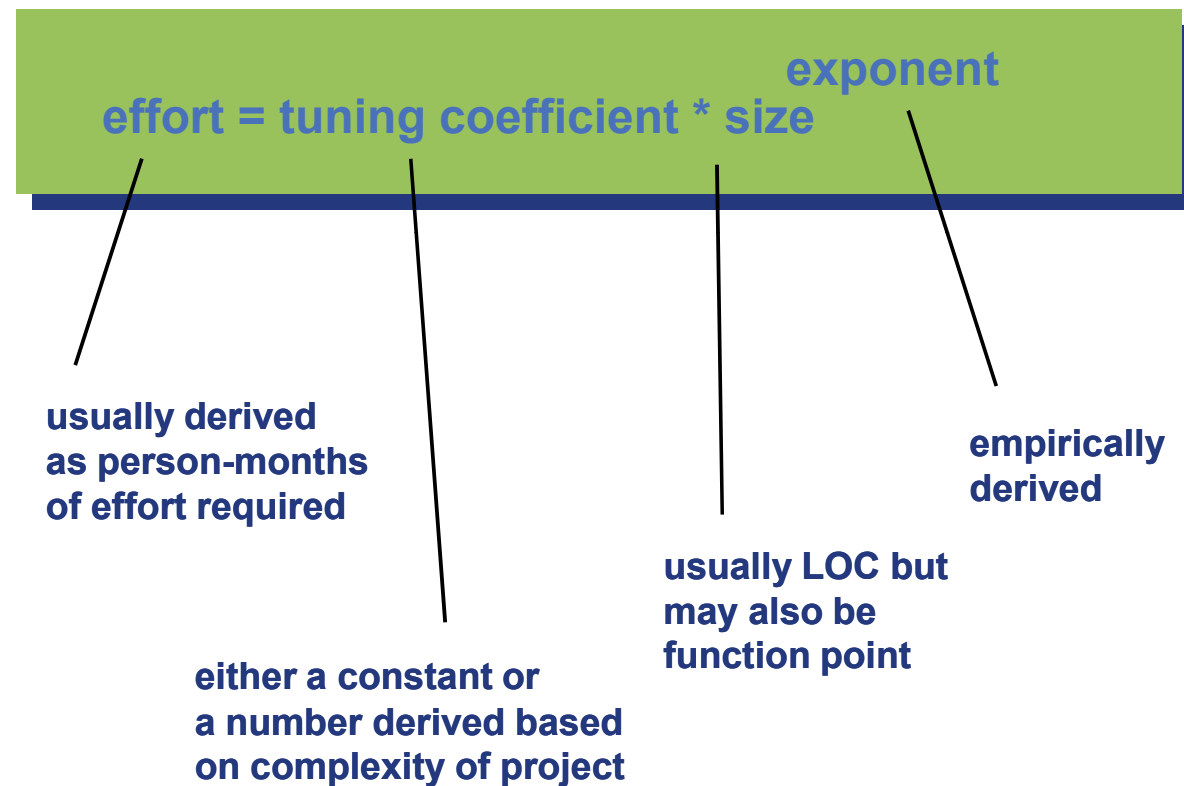
Tool-Based Estimation

基于工具的估算



Empirical Estimation Models 经验估算模型

General form:



工作量 = $A + B * (e_V)^C$ (其中 A 、 B 、 C 是经验常数, e_V 是 LOC 或 FP)

COCOMO (构造性成本模型)

COCOMO (Constructive Cost Modeling)

- ❖ An empirical model based on project experience
- ❖ Long history from initial version published in 1981 (COCOMO'81) to COCOMO II
 - COCOMO'81 is derived from the analysis of 63 software projects. (Barry Boehm, *Software Engineering Economics*, 1981)
 - COCOMO II takes into account different approaches to software development, reuse, etc.
- ❖ Very widely used models
- ❖ Several support tools available

- 基于项目经验的经验模型
- 从 1981 年 (COCOMO'81) 发布的初始版本到 COCOMO II, 历史悠久
- COCOMO'81 源于对 63 个软件项目的分析。 (Barry Boehm, 《软件工程经济学》1981)
- COCOMO II 考虑了不同的软件开发、重用等方法
- 非常广泛使用的模型
- 有多种支持工具可用

Why Are Projects Late?

- ❖ an **unrealistic deadline** established by someone outside the software development group
- ❖ **changing customer requirements** that are not reflected in schedule changes;
- ❖ an **honest underestimate** of the amount of effort and/or the number of resources that will be required to do the job;
- ❖ **predictable and/or unpredictable risks** that were not considered when the project commenced;
- ❖ **technical difficulties** that could not have been foreseen in advance;
- ❖ **human difficulties** that could not have been foreseen in advance;
- ❖ **miscommunication** among project staff that results in delays;
- ❖ a failure by project management to recognize that **the project is falling behind schedule and a lack of action to correct the problem**

Why Are Projects Late?

1. 项目为什么会延期

- 不切实际的截止日期
- 需求的变化
- 低估工作量
- 不可预测的风险
- 技术难题
- 人为困难
- 沟通不畅
- 未能认识到项目落后于计划或缺乏纠正问题的行动

Scheduling Principles

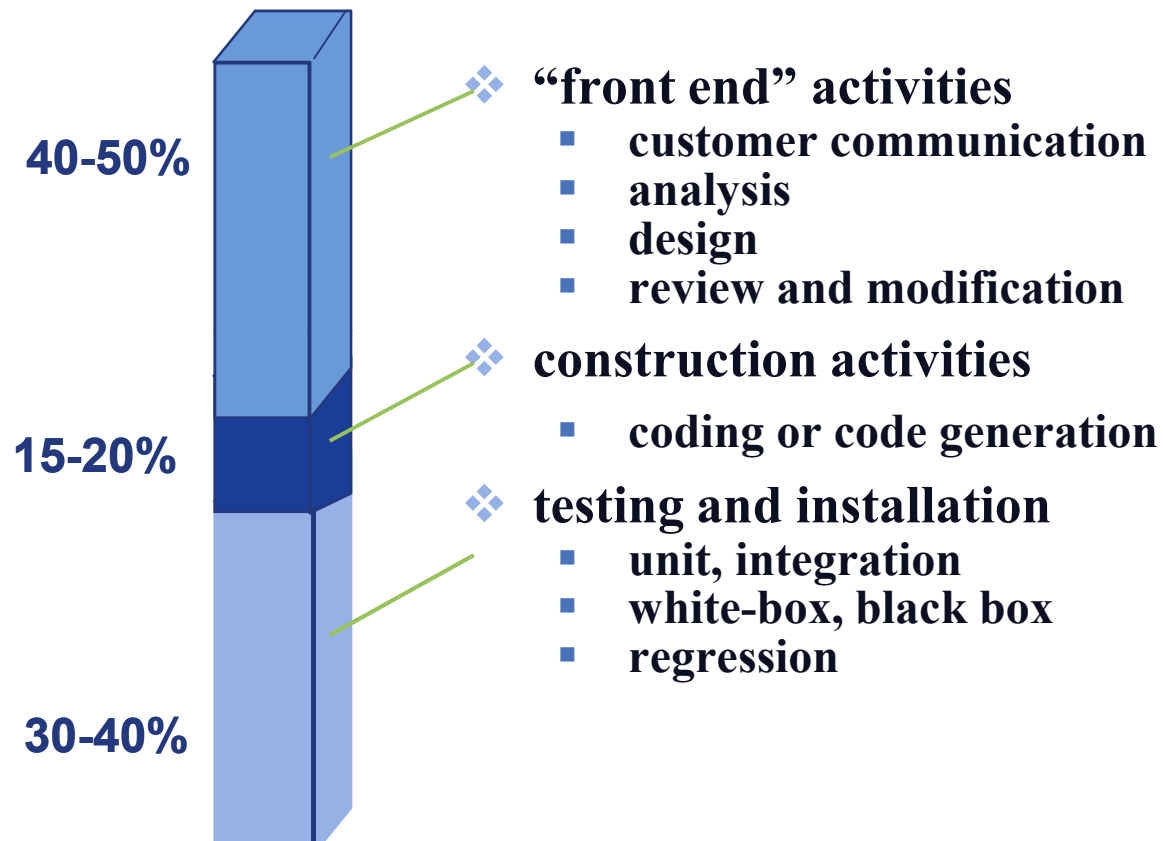
- ❖ **compartmentalization**—define distinct tasks
- ❖ **interdependency**—indicate task interrelationship
- ❖ **effort validation**—be sure resources are available
- ❖ **defined responsibilities**—people must be assigned
- ❖ **defined outcomes**—each task must have an output
- ❖ **defined milestones**—review for quality

Scheduling Principles

2. 调度的原则

- 划分——定义多个不同的任务
- 相互依赖性——明确任务的相互关系
- 工作量确认——确保人力资源可用
- 确定责任——明确责任承担者
- 明确输出结果——确定活动产生的结果
- 确定里程碑——进行质量审查

Effort Allocation



- 前端活动：客户沟通、分析、设计、评审和修改（40-50%）
- 建设活动：编码或代码生成（15-20%）
- 测试和安装：单元、集成、白盒、黑盒、回归（30-40%）

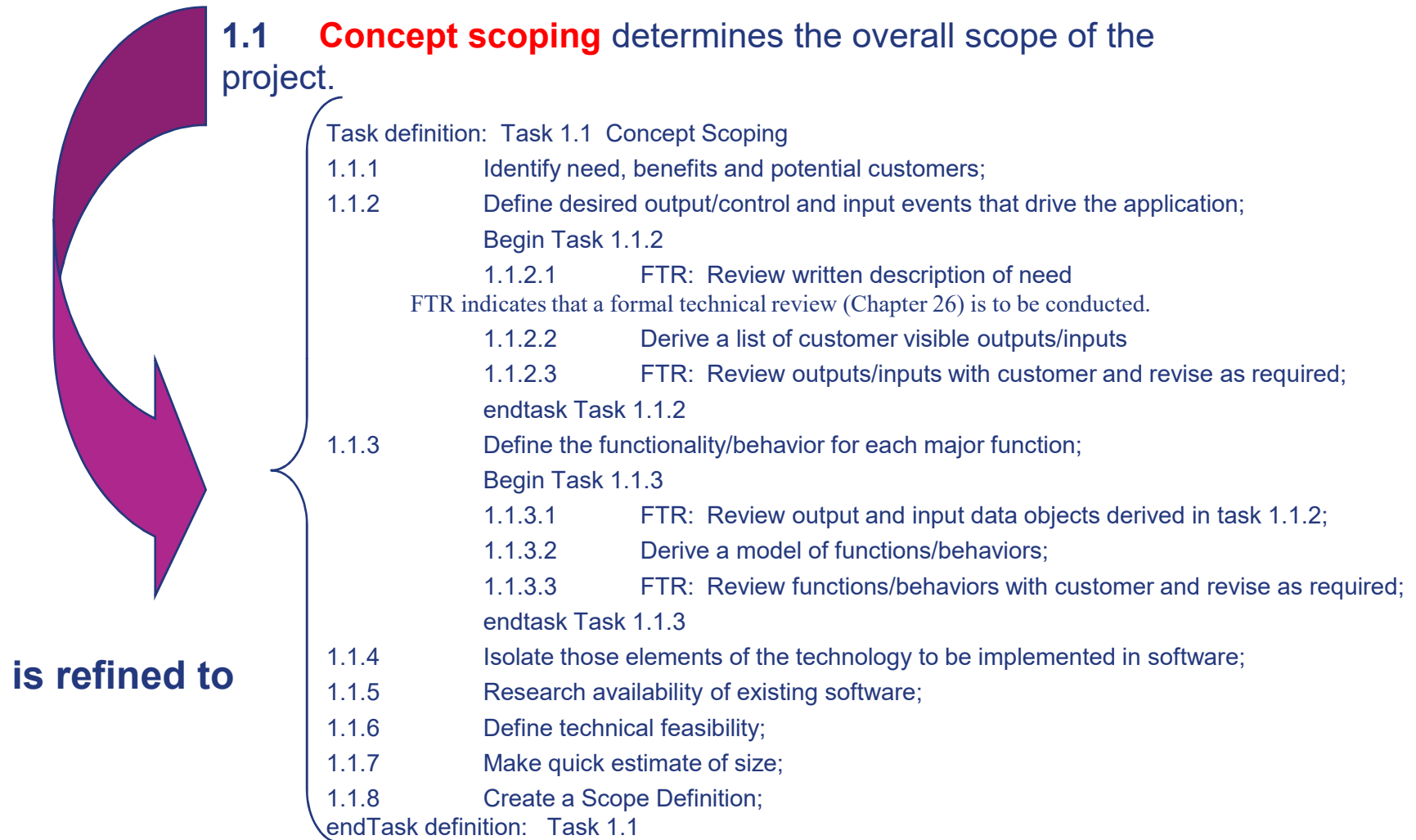
Scheduling Steps

- ❖ 1) Defining Task Sets——WBS
- ❖ 2) Sequencing Activities
- ❖ 3) Drawing Project network diagrams
- ❖ 4) Critical path analysis
- ❖ 5) Using Gantt charts (甘特图) for scheduling
- ❖ 6) Schedule Tracking

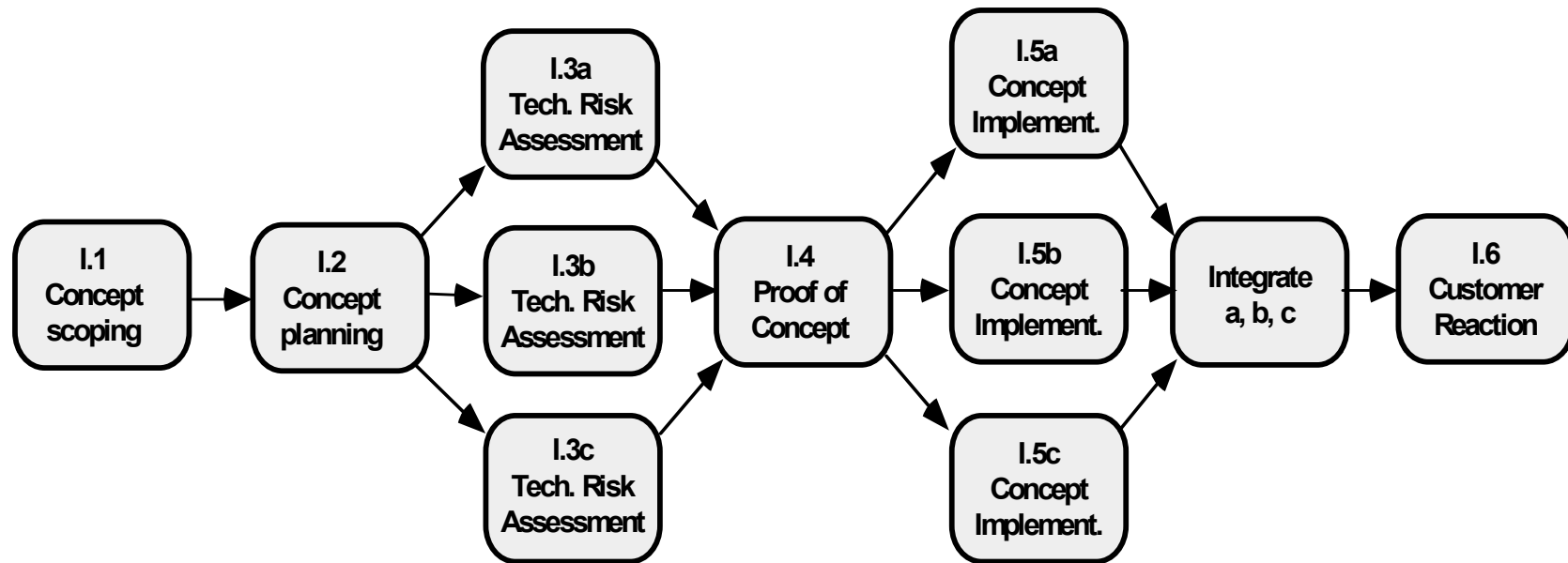
3. 调度的步骤

- | | |
|---------------|--------------|
| 1. 定义任务集——WBS | 4. 关键路径分析 |
| 2. 安排活动 | 5. 使用甘特图进行调度 |
| 3. 绘制项目网络图 | 6. 进度跟踪 |

Define Task Set



Define a Task Network



Three I.3 tasks are applied in parallel to 3 different concept functions

Three I.5 tasks are applied in parallel to 3 different concept functions

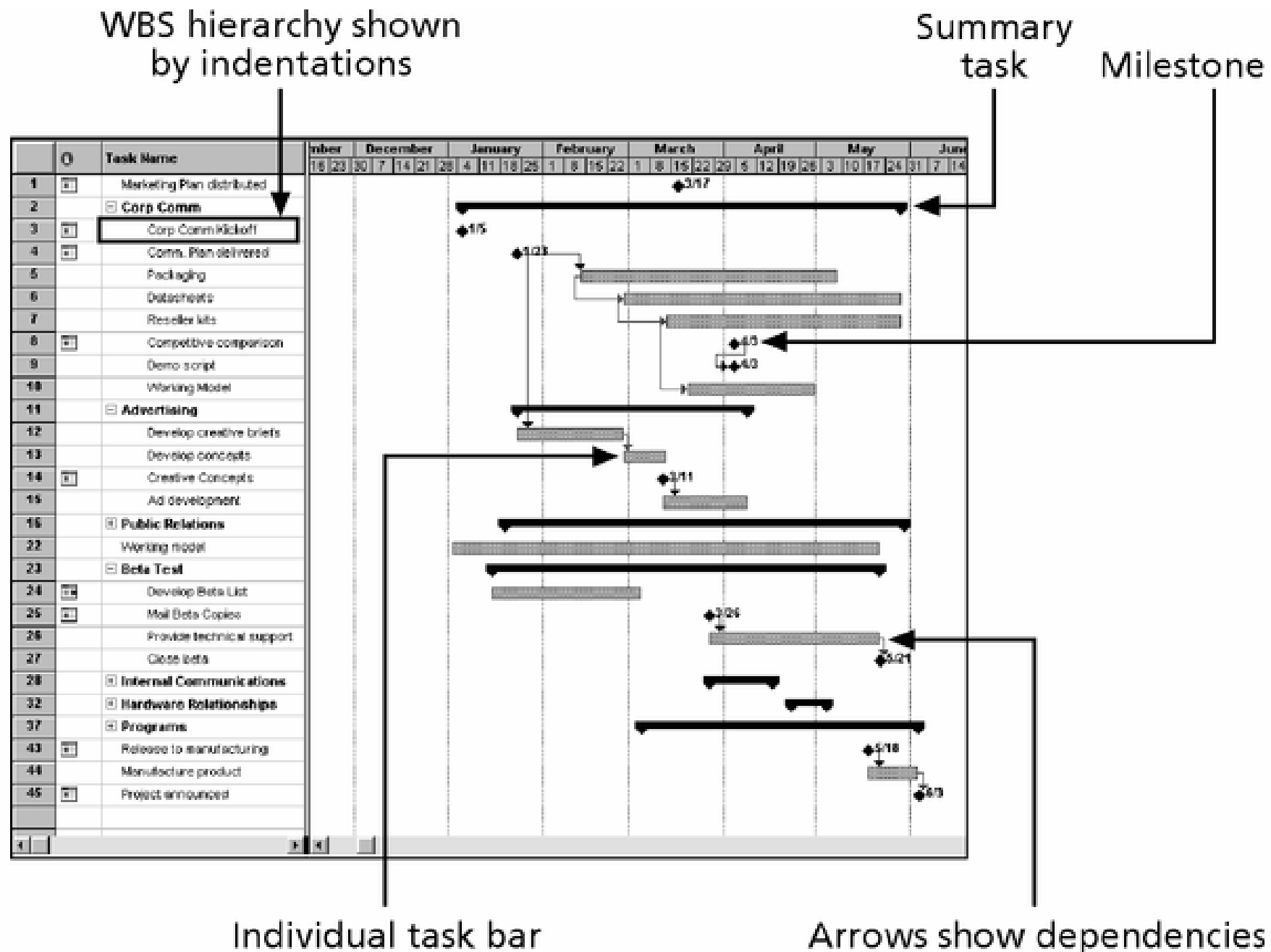
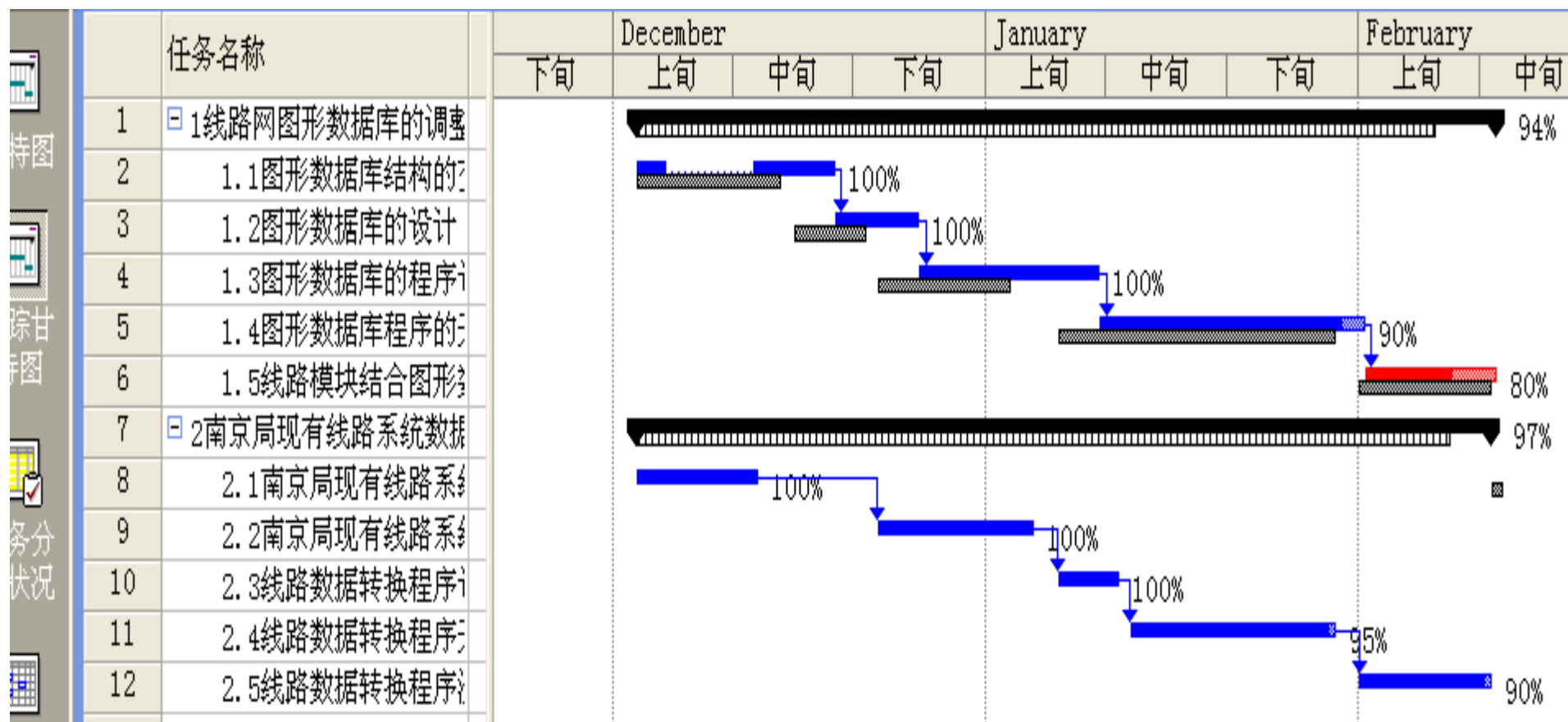


Figure 6-6. Gantt Chart for Software Launch Project



Schedule Tracking

- ❖ conduct **periodic project status meetings** in which each team member reports progress and problems.
- ❖ **evaluate the results of all reviews** conducted throughout the software engineering process.
- ❖ determine whether formal **project milestones** have been accomplished by the scheduled date.
- ❖ compare actual start-date to planned start-date for each project task listed in the resource table.
- ❖ **meet informally with practitioners** to obtain their subjective assessment of progress to date and problems on the horizon.
- ❖ use earned value analysis (Section 34.6) to **assess progress quantitatively**.

进度跟踪

- 召开定期项目状态会议，每个团队成员报告进度和问题
- 评估软件工程过程中的所有评审结果
- 确定是否按计划日期完成正式项目里程碑
- 将每个项目任务实际开始日期与计划开始日期进行比较
- 非正式地与实践者会面，获取他们对迄今为止进展的主观评估以及即将出现的问题
- 使用挣值分析（第 34.6 节）定量评估进展

Thank You !

lliao@seu.edu.cn